Supplementary Information for Ensemble ρ-SFCs: A Framework for Manifold-Aware, High-Dimensional Indexing

S0. Notation and Pipeline Contract

S0.1 Global Notation

The table below summarizes the recurring symbols used across the paper and SI.

Symbol	Meaning	Owner
x, z	point in the indexed domain	S0,S1
d	working dimension of one index tier	S0,S4
P, R	target region in 2D or d dimensions	S1,S6,S8
$\phi = (\theta, \sigma, n, P)$	expert-curve parameter tuple	S1
σ	closure or mixture parameter	S1
U, G_i, ρ_ϕ	uniform, target, and mixed densities	S1
$f_i, \mathcal{F}$	expert curve and ensemble of curves	S2
$w_i, \kappa_\mathcal{F}$	expert weight and global metric	S2
$c(f, P)$	cluster number	S6
T	recipe tree	S3
d_f	hierarchical-key fuzziness depth	S5
$\mu(P, L)$	membership of point P in child L	S5
M_{eff}	effective vocabulary after quantization	S9

S0.2 Pipeline Contract

The Ensemble ρ-SFC framework is the final indexing stage in a larger data-processing pipeline.

1. **High-dimensional clustering.** An upstream process discovers a hierarchy of semantically meaningful regions. Proteus is the intended companion mechanism, but the indexing framework can consume clusters from other sources when they provide centroids, covariance summaries, memberships, and hierarchy metadata.

2. **Target density definition.** Each cluster becomes a target density. Low-dimensional geometric clusters can use convex hulls or boundary-concentrated densities. High-dimensional semantic clusters usually use statistical summaries such as a centroid and covariance matrix.

3. **ρ-index build.** Each target density produces a compact recipe tree. Query-time synthesis combines expert trees into scalar keys or hierarchical composite keys.

This contract keeps the paper's scope clear: the framework indexes known or learned structure; it does not itself claim to discover the clustering hierarchy.

S1. Density Functions and Targets

S1.1 Mixture Definition

For an expert curve f_i, the target density is

$$\rho_{\phi_i}(z) = (1 - \sigma_i)U(z) + \sigma_i G_i(z), \qquad \sigma_i \in [0, 1).$$

The uniform component U is always present. It ensures every open subset receives nonzero mass, which is the operational condition that keeps the finite-resolution inverse key defined across the full domain. The target component G_i encodes the cluster geometry or statistics.

The parameter σ_i has two roles. At $\sigma_i = 0$, the density is exactly uniform and the generation rule returns the canonical Hilbert traversal. As σ_i approaches one, the target component dominates and the curve concentrates more traversal resolution around the cluster. The strict upper bound $\sigma_i < 1$ preserves the uniform background and prevents the expert curve from degenerating into a curve that ignores off-cluster regions.

S1.2 Low-Dimensional Geometric Targets

For 2D or 3D spatial data, a cluster often represents an object with an explicit geometric shape: a building footprint, a region boundary, a parcel, or a physical component. In those cases G_i may be concentrated near the convex hull or boundary of the region. This is the setting used in the perimeter-bound analysis of S6.

The rotation parameter θ_i is part of the curve parameterization because grid crossings depend on orientation. Choosing θ_i to align the region with the grid reduces boundary-crossing inefficiency before the recursive density allocation is applied.

S1.3 High-Dimensional Statistical Targets

For abstract embeddings, exact convex hulls are usually infeasible and often less meaningful than distributional summaries. The default target density for cluster i is therefore a multivariate Gaussian summary,

$$G_i(z) = \mathcal{N}(z; \mu_i, \Sigma_i),$$

where μ_i is the cluster centroid and Σ_i is the covariance matrix, possibly regularized or low-rank in practice. This captures center, spread, and orientation without requiring expensive high-dimensional computational geometry.

When the upstream clustering method exports fuzzy memberships, those memberships are not part of G_i itself. They are consumed later by the hierarchical composite key in S5. This separation prevents the density target and the key interpolation rule from encoding the same uncertainty twice.

S1.4 Stencil Family for Density Allocation

The mathematical definition of ρ_ϕ does not commit to a particular recursive branching scheme. The operational generator does. We fix a small family of stencils and require each index tier to commit to one. Let d be the tier dimension, b a block size with $1 \le b \le d$, and let $\pi : \{1, \dots, d\} \to \{1, \dots, d\}$ be a fixed coordinate ordering.

Orthant stencil. The orthant stencil refines a cell into all 2^d orthants in one step and stores ratios for all of them. This recovers the textbook 2^d-tree generator and, at $\sigma = 0$, the canonical d-dimensional Hilbert curve under the standard sibling order. Fanout $B = 2^d$, depth n to reach axis resolution 2^n. Use only for d small enough that B is comfortable in cache (typically $d \le 8$).

Blockwise stencil with block size b. Partition $\{1, \ldots, d\}$ into $\lceil d/b \rceil$ blocks $B_1, B_2, \ldots$ according to π. A node at depth j refines the active block $B_{(j \bmod \lceil d/b \rceil)+1}$ using a $2^{|B_k|}$-orthant Hilbert step; coordinates outside the active block are unchanged at that level. Fanout $B = 2^b$, depth to reach axis resolution 2^n is $n \cdot \lceil d/b \rceil$. At $\sigma = 0$ this recovers a blockwise Hilbert product, which is the tier-local canonical curve H_{tier} for the chosen b and ordering π.

Sparse stencil with uniform remainder. At each node, select a subset $S \subseteq \{1, \ldots, 2^d\}$ of active children with $|S| \leq B_{\max}$. Compute integrated target masses for those children; lump the complement under one uniform-remainder branch whose mass equals the residual target plus the full uniform-baseline mass of those orthants. At $\sigma = 0$, the remainder mass is exactly uniform, so the stencil reduces to the canonical blockwise or orthant baseline that the implementation uses to traverse off-stencil children. The sparse stencil is therefore an approximation: the recipe tree encodes a truncated density $\hat{\rho_\phi}$ that agrees with ρ_ϕ on S and is uniform on S^c. The pointwise error is bounded by the off-stencil target mass, which is small whenever the target is well-concentrated.

The tier-local canonical curve is denoted H_{tier} in the rest of the SI. The classical Hilbert curve is the special case where the tier uses the orthant stencil.

S2. Global Metric Properties

Let $\mathcal{F} = \{f_i\}_{i=1}^N$ be an ensemble of finite-resolution expert curves and let $\sum_i w_i = 1$. The operational inverse key for expert i is denoted $K_i(x) = f_i^{-1}(x)$, with ties resolved by the implementation's fixed cell-order convention. The global key is

$$\kappa_{\mathcal{F}}(x) = \sum_{i=1}^{N} w_i K_i(x).$$

S2.1 Finite-Resolution Continuity Convention

At finite depth n, each K_i is a key over cells of side length 2^{-n}. The implementation uses the cell containing x and returns the associated time interval midpoint or interval coordinate. Under this finite-resolution convention, the key is stable under perturbations that do not cross a cell boundary, and boundary behavior is controlled by the common tie-breaking rule. As n increases, the key becomes the limiting inverse-coordinate representation used by the index.

This is the object meant by the global metric in the system architecture. A topological inverse of a continuous space-filling curve from $[0, 1]$ to $[0, 1]^d$ is not the operational object used by the index. The index uses finite-depth inverse keys and their limiting refinements.

S2.2 Tier-Local Canonical Curve Base-Case Limit

Let the tier commit to a stencil from S1.4 with tier-local canonical curve H_{tier}. Assume each expert generator in the tier satisfies

$$\lim_{\sigma_i \to 0} K_i(x) = H_{\text{tier}}^{-1}(x)$$

for every point x outside implementation tie boundaries, where H_{tier}^{-1} is the inverse key of the tier-local canonical curve at the corresponding resolution. Then

$$\lim_{\forall i, \sigma_i \to 0} \kappa_{\mathcal{F}}(x) = \lim_{\forall i, \sigma_i \to 0} \sum_{i=1}^{N} w_i K_i(x) = \sum_{i=1}^{N} w_i H_{\text{tier}}^{-1}(x) = H_{\text{tier}}^{-1}(x).$$

Thus the ensemble smoothly reduces to the tier-local canonical ordering when every expert is returned to the uniform-density case. With the orthant stencil this is the classical Hilbert key.

S2.3 Weighted Consensus Interpretation

The scalar key $\kappa_{\mathcal{F}}$ is best understood as a consensus coordinate rather than a replacement for exact high-dimensional distance. Query-time weights can emphasize one cluster family, de-emphasize another, or revert toward the canonical baseline. Because expert lookups are independent, the consensus can be evaluated in parallel and adjusted without rebuilding the recipe trees.

S3. Recipe Tree Storage Layout

S3.1 Node Format

Each expert curve stores a recipe tree T in level-order layout. The important implementation invariant is bounded local branching. A literal 2^d fanout is acceptable only for very small d; at $d = 32$ it would require more than four billion child ratios at one warped node and is not the intended data structure. Instead, each node owns a local branching stencil of size B, commonly $B = 2^b$ for a small block dimension $b \leq 8$, or a sparse set of active children plus a uniform remainder. A node begins with a tag byte:

- WarpedTag: the node stores B floating-point ratios for its local branching stencil. The ratios sum to one and partition the current 1D time interval among those branches.
- UniformTag: the node and all descendants are uniform. No ratio payload is stored, and lookup switches to the canonical fast path for the tier, such as a Hilbert key or blockwise Hilbert product.

S3.2 Build Protocol

The build protocol recursively integrates the target density over the node's bounded local branching stencil. If the branch masses are all within a uniformity tolerance of $1/B$, the node is emitted as UniformTag. Otherwise, the normalized branch masses are emitted as ratios under WarpedTag, and the build recurses into branches whose density remains nonuniform or whose target error exceeds tolerance.

S3.3 Query Protocol

The query protocol maintains a current 1D interval. At a warped node, it determines which child cell contains the point, adds the ratio mass of earlier children to the time coordinate, scales the interval by the selected child ratio, and descends. At a uniform node, it computes the remaining Hilbert suffix by the integer fast path and returns immediately.

S3.4 Synthesis

Given N recipe trees, synthesis evaluates all N independent lookups and computes

$$\kappa_{\mathcal{F}}(x) = \sum_{i=1}^{N} w_i \, \textsc{QueryRecipeTree}(x, T_i, n).$$

Only the weights need change for most query-time metric adjustments. If an expert's density target changes, only that expert's recipe tree needs to be rebuilt.

S3.5 Stencil Invariants and Consequences

Choosing a stencil from S1.4 is operational, but it carries four invariants that the rest of the framework depends on.

Algorithm 1 BUILDRECIPETREE: density-ratio recipe construction

procedure BUILDRECIPETREE($cell, depth, n, \rho_\phi, stencil$)
  if $depth = n$ **then**
    return leaf marker
  end if
  Generate the bounded branch cells specified by $stencil(cell, depth)$
  for each branch q **do**
    $mass[q] \leftarrow \int_{branch(q)} \rho_\phi(z)\, dz$
  end for
  $ratios \leftarrow mass / \sum_q mass[q]$
  if *ratios* are uniform within tolerance **then**
    Emit UNIFORMTAG
  else
    Emit WARPEDTAG and *ratios*
    for each branch q **do**
      BUILDRECIPETREE($branch(q), depth + 1, n, \rho_\phi, stencil$)
    end for
  end if
end procedure

Shared stencil per tier. $\kappa_{\mathcal{F}}$ is the weighted sum of inverse keys $K_i(x)$. These keys are only comparable when they index space along the same recursive partition with the same sibling ordering. The tier's index schema must therefore fix $(b, \pi,$ sparse-stencil parameters) once at build time. All expert curves in that tier are built and queried against that schema.

Redefined base case. The $\sigma = 0$ limit produces the tier-local canonical curve H_{tier}, not necessarily the classical orthant Hilbert curve. The uniform fast path therefore evaluates the canonical key for the chosen stencil. With the orthant stencil this is the standard Hilbert key; with a blockwise stencil it is the blockwise Hilbert product. The proofs of S2 are stated in this generalized form.

Sparse-stencil approximation. A sparse stencil with uniform remainder encodes a truncated density $\hat{\rho_\phi}$, equal to ρ_ϕ on the active set S and uniform on S^c. The pointwise error is bounded by the off-stencil target mass. The Hilbert-limit property is unaffected because $\rho_\phi = U$ on S^c at $\sigma = 0$. The worst-case and average-case cluster bounds remain valid if ρ_ϕ is replaced by $\hat{\rho_\phi}$; bound constants depend on the truncation budget.

Depth multiplier and uniformity granularity. With block size b a query visits $n \cdot \lceil d/b \rceil$ levels to reach axis resolution 2^n, so per-curve cost is $O(n \cdot \lceil d/b \rceil)$. Uniformity detection at a warped node tests B block-marginal ratios against $1/B$. A node can therefore be marked UNIFORMTAG in the block-marginal sense even when joint density across blocks is not uniform; this is by design and gives the fast path additional opportunities to fire.

S4. Hierarchical Cascade

S4.1 Setup

Choose target tier dimensions $d_1 < d_2 < \cdots < d_L$, such as $8, 16, 32$. For each tier ℓ, fit a projection P_ℓ from the full representation to $\mathbb{R}^{d_\ell}$, choose a stencil from S1.4, and build a complete ρ-index on the projected data. The tier dimension is not the recipe-tree fanout dimension: a 32D tier

Algorithm 2 QUERYRECIPETREE: full lookup protocol

```
procedure QueryRecipeTree(x, T, n)
    time ← 0; interval ← 1; node ← 0
    for depth ← 0 to n − 1 do
        if T[node].tag = UniformTag then
            suffix ← HilbertSuffix(x, depth, n, d)
            return time + interval · suffix
        end if
        q ← BranchIndex(x, T[node], depth)
        r ← T[node].ratios
        time ← time + interval ∑_{j<q} r[j]
        interval ← interval · r[q]
        node ← ChildIndex(node, q)
    end for
    return time
end procedure
```

is represented by bounded-arity or blockwise recipe nodes, not by a single 2^{32}-child split. The projections are computational devices; final fidelity is recovered by reranking in the original space.

S4.1.1 Tier Stencil Selection

The two relevant quantities for a tier of dimension d_ℓ with block size b_ℓ are the per-node ratio table size $B_\ell = 2^{b_\ell}$ and the depth multiplier $\lceil d_\ell / b_\ell \rceil$. Storage scales like B_ℓ per warped node; traversal cost scales like $\lceil d_\ell / b_\ell \rceil$. The heuristic default is:

- $d_\ell \leq 8$: orthant stencil, $b_\ell = d_\ell$, $B_\ell = 2^{d_\ell} \leq 256$, depth multiplier 1.
- $8 < d_\ell \leq 32$: blockwise stencil, $b_\ell = 8$, $B_\ell = 256$, depth multiplier $\lceil d_\ell / 8 \rceil \in \{2, 3, 4\}$. At $b_\ell = 8$ one warped node fits in roughly one cache line (256 floats = 1KB), which keeps the fast path cache-resident.
- $d_\ell > 32$: $b_\ell \in \{4, 5, 6\}$, $B_\ell \in \{16, 32, 64\}$, depth multiplier $\lceil d_\ell / b_\ell \rceil$. Alternatively a sparse stencil whose active-set size is bounded by a tunable $B_{\max}$.

These are defaults, not theorems. Implementations should log (b_ℓ, π_ℓ) and the empirical fast-path fire rate; if uniform regions dominate, raising b_ℓ trades storage for shorter traversals. The recommended values reflect the constraints $B_\ell \leq 256$ (fits in one cache line for float ratios), depth multiplier $\leq d_\ell / 4$ (keeps query cost within roughly $4\times$ the orthant baseline at the same n), and a soft preference for $b_\ell \geq 4$ so that block-marginal uniformity remains informative.

S4.2 Query Funnel

For a query point q, Tier 1 searches the broadest projected index and returns an expanded candidate set of size k_1. Later tiers rerank only the surviving candidates and keep k_ℓ items, typically with $k_1 \gg k_2 \gg k$. The final step retrieves full-fidelity vectors and returns the top k under the desired exact distance.

S4.3 Candidate Sizing

Candidate sizes are workload-dependent. The paper's Wikipedia example uses an illustrative funnel: 7.5B items to about 1M, then about 10,000, then $k = 200$, followed by exact reranking.

Algorithm 3 CascadeQuery: SI-level cascade specification

procedure CascadeQuery(q, k)
  $C_1 \leftarrow$ RhoIndexKNN($I_1, P_1 q, k_1$)
  for $\ell \leftarrow 2$ **to** L **do**
    Score each $c \in C_{\ell-1}$ by $\|P_\ell X_c - P_\ell q\|$
    $C_\ell \leftarrow$ top k_ℓ scored candidates
  end for
  Score each $c \in C_L$ by full-fidelity distance $\|X_c - q\|$
  return top k candidates
end procedure

These numbers are not guarantees; they are the operating targets that make the feasibility estimate concrete.

S4.4 Cost of the High-Dimensional Tier

Stencilling does not save memory or time relative to an orthant-Hilbert recipe tree at $d = 32$, because that baseline is not realizable: one orthant warped node at $d = 32$ would store $4 \cdot 2^{32} \approx 17\,\text{GB}$ of ratios. The architectural value of the stencil family is that it allows the cascade to include a tier with $d > 10$ at all. This subsection records the per-query budget that justifies including such a tier.

Marginal value of a 32D tier. Without a 32D tier, the cascade is 8D $\to$ 16D $\to$ rerank, and roughly 10^4 candidates reach the exact-distance stage. With a 32D tier, the cascade is 8D $\to$ 16D $\to$ 32D $\to$ rerank, and roughly $k = 200$ candidates reach exact distance. The saving is approximately $10^4 - 200 \approx 9{,}800$ exact distance evaluations per query.

Cost of an exact 768D distance. A 768D float16 distance is about 1,500 floating-point operations. With SIMD, throughput is typically $1{-}2\,\mu$s per pair. Rerank of 10^4 candidates therefore costs about $10{-}20\,$ms per query; rerank of 200 candidates costs about $0.2{-}0.4\,$ms. The added tier saves $\sim 10\,$ms per query at the rerank stage.

Cost of a 32D recipe-tree query with $b = 8$. At axis precision $n = 8$ bits and block size $b = 8$, one expert curve visits at most $n \cdot \lceil d/b \rceil = 8 \cdot 4 = 32$ bounded-arity nodes per query. Each uniform-tag node is a one-byte read; each warped node scans a 1 KB ratio table. Worst-case memory traffic per expert is $\sim 32\,$KB, which fits in L2 cache and runs in $\sim 1{-}10\,\mu$s. With the fast path dominating in regions far from the expert's target, the typical cost is much smaller.

Ensemble cost. For an ensemble of N experts queried at the 32D tier, the per-query cost is $\sim N \cdot 1{-}10\,\mu$s serially, e.g. $\sim 0.1{-}1\,$ms at $N = 100$. Independent expert lookups are embarrassingly parallel, so on a GPU or with SIMD batching the wall-clock cost is closer to $1{-}10\,\mu$s even at large N.

Budget summary. The 32D tier saves on the order of 10 ms per query in rerank work and costs on the order of $0.01{-}1\,$ms per query in recipe-tree traversal, a one- to three-order-of-magnitude net win. The block-size knob b controls how this cost trades against per-node ratio storage: $b = 8$ keeps each warped node within a single cache line, $b = 4$ reduces per-node storage by $16\times$ at the cost of doubling traversal depth. The point is that this entire budget exists only because stencilling makes the 32D tier representable in the first place.

S5. Hierarchical Composite Key Derivation

S5.1 Why Values Need Namespaces

A composite key cannot be a simple concatenation of SFC values. A value from an animal-cluster curve is not directly comparable to a value from a plant-cluster curve unless the key says which cluster supplied the value. The atomic component is therefore a pair: a child identifier and a value.

A fixed global cluster identifier would solve comparability but would not encode hierarchy. The preferred identifier is a variable-length path: at each level, store the child index within its parent. This makes the key self-describing and makes subtree queries natural.

S5.2 Centroid Prefix

At parent cluster C, child clusters should be ordered by a property of the child as a whole, not by arbitrary individual points. The stable anchor is the child centroid. A sharp prefix for a point P in child L therefore uses

$$f_C^{-1}(\text{centroid}(L)).$$

This creates robust blocks but also creates hard boundaries between siblings.

S5.3 Membership-Weighted Fuzzy Value

To soften those boundaries, the value component for a configurable number of upper levels is

$$\text{FuzzyVal}_C(P) = \mu(P, L) f_C^{-1}(\text{centroid}(L)) \; + \; \big(1 - \mu(P, L)\big) f_C^{-1}(P).$$

Core points with $\mu(P, L) \approx 1$ remain dominated by the centroid and keep the cluster block stable. Boundary or outlier points with smaller memberships are allowed to move toward their own geometric location.

S5.4 Fuzziness Depth

The fuzziness depth d_f controls how many levels above the leaf use the fuzzy value. At $d_f = 0$, keys are sharp and highly compressible. At $d_f = 1$, the most common boundary problem is addressed with one extra SFC traversal per point during build. Higher d_f values increase fidelity but also increase key entropy and build cost.

S6. Worst-Case Clustering Bound

S6.1 Statement

For every convex polygon $P \subset [0, 1]^2$ with perimeter p, there exists a parameter setting $\phi^* = (\theta^*, \sigma^*, n, P)$ such that

$$c(f^{(\phi^*)}, P) \leq 2 + \left\lceil \frac{p\, 2^n}{8} \right\rceil.$$

S6.2 Proof Sketch

The parameter choice is constructive at the level of the density target.

1. **Target selection.** Set the target region to P itself and choose θ^* to align the polygon with the grid in the orientation that minimizes avoidable boundary crossings.

2. **Closure.** Choose $\sigma^* = 1 - \epsilon$ for small $\epsilon > 0$. The target density then concentrates the curve in a narrow band around the polygon while retaining a uniform background.

3. **Boundary crossing count.** For a fixed depth n, the grid has cell width 2^{-n}. A traversal that is attracted to the boundary of P can align its entries and exits with the boundary band. The number of such crossings is controlled by the perimeter measured in grid units.

4. **Constant-factor improvement.** Relative to a generic Hilbert traversal, the target density removes the main orientation mismatch. The resulting bound replaces the generic perimeter constant with the improved 1/8 factor, plus the two endpoint intervals that cover entrance and exit effects.

This is a worst-case geometric bound. It does not claim that typical clusters require this many intervals; S7 gives the average-case contiguity argument.

S7. Probabilistic Argument for $c = 1$

The deterministic bound in S6 must hold for adversarial or highly elongated polygons. For typical clusters, the expected cluster number should often be one.

Error budget. As $\sigma \to 1$, the density forces most traversal mass into a tube T around the target region. Let ϵ be the fraction of curve length allowed outside this tube. By choosing σ close enough to one, ϵ can be made small while preserving the uniform background.

Cause of non-contiguity. A cluster number greater than one requires an in-out-in event: the curve enters the cluster, leaves it, and re-enters later. Such an event must be supported by off-tube segments, because the dominant in-tube path is aligned with the target.

Probability heuristic. Let the total curve length be L. The off-tube length is ϵL, while the target-aligned length is $(1 - \epsilon)L$. For the small off-tube path to sever the contiguity of the dominant path, several unlikely placements must coincide. The probability of a segment being off-tube is proportional to ϵ, and the probability of a sequence that creates an in-out-in split is a higher-order function of ϵ. Thus, for any confidence level $1 - \alpha$, the model expects a sufficiently high $\sigma < 1$ to make $\Pr[c > 1] < \alpha$ under non-adversarial generation assumptions.

S8. Generalization to d Dimensions

S8.1 Recursive Domain

At the theoretical level, the generation process can be described as extending from quadtrees to 2^d-trees over $[0, 1]^d$, where the density mass in each child hypercube determines the time-interval split. This is a mathematical existence model, not the high-dimensional storage layout. Operational high-dimensional tiers approximate the same density allocation with bounded-arity blockwise, binary, or sparse stencils. The strict positivity of ρ_ϕ for $\sigma < 1$ gives every open set nonzero allocation at every finite resolution under either description.

S8.2 Surjectivity and Continuity

At the continuous idealization level, the recursive construction has dense image in $[0, 1]^d$ because every open cell receives nonzero time allocation. The image of a continuous map from compact $[0, 1]$ is compact. In the Hausdorff domain $[0, 1]^d$, a compact dense subset is the full space, so the idealized curve is surjective. Continuity follows from the recursive connection rule that joins neighboring child traversals without jumps.

S8.3 Clustering Bound Intuition

In d dimensions, the boundary term controlling cluster fragmentation is the $(d-1)$-dimensional surface area $A(\partial R)$. A density concentrated near ∂R aligns the traversal with that boundary and should therefore improve the constant in a surface-area-proportional bound. The exact constant is not derived here; the important point is that the same boundary-alignment principle carries over while its relative geometric advantage shrinks as dimension grows.

The constant in such a bound also depends on the chosen stencil from S1.4. The idealized orthant model gives the strongest geometric advantage, while blockwise and sparse stencils introduce additional grid-aligned crossings between blocks and pay a stencil-dependent constant. The qualitative claim, that boundary-aligned density yields a surface-area-proportional cluster count with a better constant than a generic curve at the same stencil, survives in all variants.

S9. Wikipedia Feasibility Derivation

Status. This section is a preliminary theoretical analysis intended to establish plausible feasibility for a large-scale scenario. The empirical study and implementation have not yet been performed.

S9.1 Baseline Parameters

- Dataset: English Wikipedia text corpus.
- Total tokens: $M = 7.5\text{B}$.
- Embedding model: 768D vectors with float16 precision.
- Raw vector size: $7.5\text{B} \times 768 \times 2$ bytes $= 11.52\text{TB}$.

S9.2 Effective Vocabulary

Language embeddings are not uniformly random points. Token repetition, semantic clustering, projection to 32D, and 8-bit quantization cause many tokens to share quantized coordinates. The working assumption is an effective vocabulary

$$M_{\text{eff}} = 500\text{M}$$

unique quantized points.

S9.3 Leaf Information Bound

The minimum number of bits required to distinguish one of M_{eff} items is

$$\log_2(500 \times 10^6) \approx 29.$$

Thus the leaf information content is

$$500\text{M} \times 29 \text{ bits} = 14.5 \times 10^9 \text{ bits} \approx 1.81\text{GB}.$$

S9.4 Tree Overhead and Tier Total

Assume the adaptive recipe-tree overhead is $1.5\times$ the leaf information size. A single tier is then

$$1.81\text{GB} \times 1.5 = 2.72\text{GB}.$$

With three tiers, the full cascade estimate is

$$3 \times 2.72\text{GB} = 8.16\text{GB}.$$

This is below 10GB and corresponds to about 0.07% of the 11.52TB raw vector store.

S9.5 Hierarchical-Key Caveat

The estimate above assumes a generic locality-preserving ordering. Hierarchical keys depend on d_f. Sharp keys ($d_f = 0$) should compress strongly because prefixes repeat. The recommended $d_f = 1$ should remain near the 8–10GB estimate. Larger d_f values increase entropy and may increase index size, though the fixed recipe-tree cost remains the dominant term.

S10. Relationship to Wavelet Trees

The recipe-tree system and wavelet-tree backend have complementary roles. The Ensemble ρ-SFC framework is the semantic sorter: it maps high-dimensional points to 1D keys whose order reflects learned cluster structure. A wavelet tree is the compressed payload index: once items are sorted by semantic key, it supports rank, select, quantile, and range-style analytical queries over payload fields.

The hierarchical composite key makes this relationship stronger. Because a parent cluster and its descendants occupy a contiguous key range, a wavelet-tree query can be restricted to a semantic subtree by targeting that key interval. For example, a system could ask for a quantile statistic over all items in a learned cluster and its descendants using the same compressed sequence operations used for global queries.